Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes
Département Ingénierie des Systèmes Embarqués
Filière Smart Systems Engineering

Tutorial 4 : Écouter le moteur avec l'Arduino Portenta H7 : audio, FFT et détecteur d'anomalie embarqué dans OpenMV IDE

Séries de tuto sur Portenta H7, lecture 4 « Écouter la machine »

Pr.Hassan BERBIA/Pr.Aissam BERRAHOU

2 novembre 2026

Objectifs du Tutorial

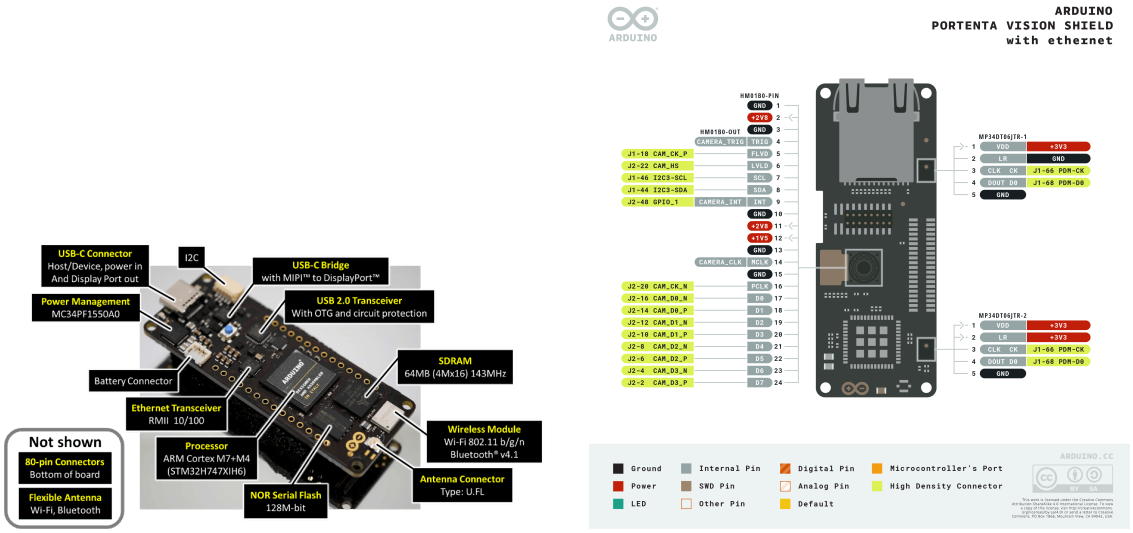
Dans ce tutoriel, la Portenta H7 devient l'oreille de la station EcoTri 4.0. Vous allez acquérir le son des deux micros du Vision Shield, l'enregistrer en WAV sur microSD, calculer un spectre avec `ulab`, extraire des descripteurs (RMS, kurtosis, énergies par bande) puis construire, entièrement en MicroPython, un **détecteur d'anomalie** qui apprend le bruit normal du moteur et alerte quand il change. Chaque étape est reliée au cours (Nyquist, FFT, fenêtrage, bandes mel, seuil). À la fin de ce tutoriel, vous serez capable de :

- Configurer le module `audio` d'OpenMV (fréquence, gain, filtre passe-haut) et recevoir les échantillons par rappel.
- Mesurer le niveau d'un signal, détecter la saturation et choisir le gain.
- Enregistrer des fenêtres audio en fichiers WAV sur la carte microSD.
- Calculer une FFT fenêtrée (Hann) et des énergies par bande avec `ulab`.
- Construire un détecteur d'anomalie embarqué (apprentissage du normal, score, seuil, lissage).
- Mesurer le temps de calcul de chaque étape et le comparer au budget du cours.

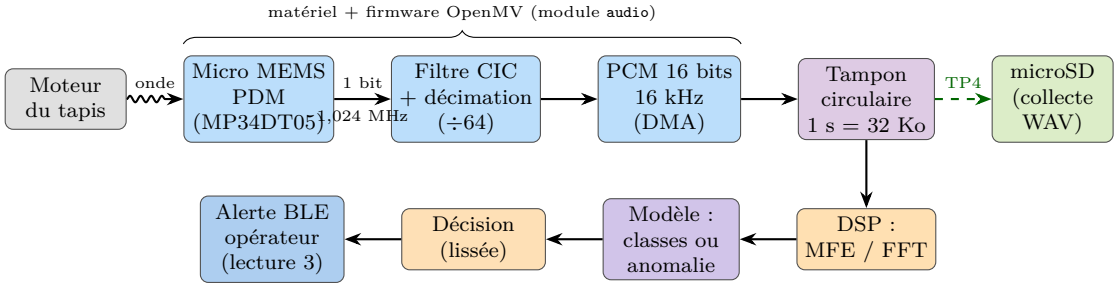
Durée indicative : 1 h 15. Ce tutoriel prépare le TP4 (dataset moteur, Edge Impulse et jalon J4).

1 Introduction : le son sur la Portenta H7

Le **Vision Shield** de la Portenta H7 porte, en plus de la caméra, deux microphones MEMS omnidirectionnels **MP34DT05** (STMicroelectronics) à sortie **PDM**. Le STM32H747 reçoit leur flux d'un bit, le filtre et le décime ; le firmware OpenMV livre au script MicroPython des blocs d'échantillons PCM 16 bits par l'intermédiaire du module `audio`. La version Ethernet du Vision Shield possède aussi un lecteur de carte **microSD**, que nous utiliserons pour la collecte.



La Portenta H7 et le Vision Shield : les deux micros PDM sont situés de part et d'autre du capteur d'image ; le lecteur microSD est au dos (version Ethernet).



Rappel du cours : la chaîne d'acquisition. Ce tutoriel parcourt toutes les étapes, de l'onde sonore à l'alerte.

Ce que nous allons construire

Étape	Programme	Notion du cours
1–2	`audio_niveau.py`	PCM, gain, saturation, RMS
3	`audio_wav.py`	format WAV, collecte, métadonnées
4	`audio_fft.py`	FFT, Hann, résolution f_e/N
5	`audio_bandes.py`	bandes de type mel, log, descripteurs
6–7	`audio_anomalie.py`	apprentissage du normal, score, seuil, lissage

2 Pré-requis

Matériel

- Carte **Arduino Portenta H7** avec son **Vision Shield** (version Ethernet pour la microSD).
- Carte **microSD** (FAT32, 4 à 32 Go) et câble USB-C (données).
- Une source de bruit de moteur : le moteur du tapis, un petit ventilateur USB, ou à défaut un enregistrement de moteur joué sur un haut-parleur (jeu de données MIMII).

Logiciel

1. **OpenMV IDE (version 4.0 ou plus)** et le firmware OpenMV à jour sur la Portenta (Tutorial 1).
2. Sur le PC : **Python 3** avec `numpy`, `scipy` et `matplotlib` (vérification des fichiers WAV).
3. Documentation : modules `audio` et `ulab` sur docs.openmv.io ; exemples **File** → **Examples** → **Audio** dans OpenMV IDE.

Astuce

Insérez la microSD **avant** de mettre la carte sous tension : OpenMV la monte au démarrage. Selon la version du firmware, elle apparaît comme racine du système de fichiers (`/`) à la place de la Flash interne, ou sous `/sdcard`. Le script de l'étape 3 détecte les deux cas.

3 Étapes détaillées du tutoriel

Étape 1 : Recevoir les échantillons audio

Le module `audio` appelle une fonction de rappel à chaque bloc d'échantillons. Ce rappel est appelé par le firmware pendant l'acquisition : il doit être **très court**, sans affichage ni calcul lourd. Il copie le bloc dans un tampon et lève un drapeau ; la boucle principale fait le reste.

```
# audio_niveau.py - acquisition et niveau du signal (Portenta H7 + Vision Shield)
import audio, time, micropython
from ulab import numpy as np

FE = 16000                          # fréquence d'échantillonnage (Hz)
CANAUX = 1                          # 1 micro suffit ; 2 pour la stéréo
GAIN_DB = 24                        # à figer et à noter dans la fiche du dataset
micropython.alloc_emergency_exception_buf(100)

bloc = None
pret = False

def rappel(buf):                    # appelé par le firmware, en interruption
    global bloc, pret
    if not pret:
        bloc = np.frombuffer(buf, dtype=np.int16)
        pret = True

audio.init(channels=CANAUX, frequency=FE, gain_db=GAIN_DB, highpass=0.9883)
audio.start_streaming(rappel)

t0 = time.ticks_ms(); n = 0
```

```
while time.ticks_diff(time.ticks_ms(), t0) < 5000:
    if pret:
        x = bloc
        rms = np.sqrt(np.mean(x * x * 1.0))          # *1.0 : calcul en flottant
        crete = np.max(abs(x))
        print("bloc %3d : %4d éch., RMS = %7.1f, crête = %6d %s" %
              (n, len(x), rms, crete, "SATURATION" if crete >= 32700 else ""))
        n += 1
        pret = False
audio.stop_streaming()
print(n, "blocs en 5 s =>", n * len(x) / 5, "échantillons/s")
```

— Vérifiez que le nombre d'échantillons par seconde est proche de 16 000 : c'est votre f_e **effective**.

— Parlez, tapez sur la table, approchez la carte du moteur : observez RMS et crête.

— Le paramètre `highpass` règle le filtre qui supprime la composante continue (0,9883 à 16 kHz : fréquence de coupure de l'ordre de 30 Hz).

Question

Combien d'échantillons contient un bloc ? Quelle durée cela représente-t-il ? Le filtre passe-haut supprime-t-il la raie 1x du moteur (25 Hz) ? Quelle conséquence pour la détection du balourd, et faut-il en tenir compte dans le choix des caractéristiques ?

Étape 2 : Choisir le gain

Placez la carte à sa position de mesure, moteur en marche, et faites varier `GAIN_DB` (12, 18, 24, 30, 36). Pour chaque valeur, relevez le RMS moyen et la crête maximale sur 5 s, moteur à l'arrêt puis en marche.

Gain (dB)	RMS arrêt	RMS marche	Crête marche	Saturation ?
12				
18				
24				
30				
36				

Tableau à compléter. Retenez le plus grand gain dont la crête reste inférieure à environ 16 000 (marge de 6 dB pour les chocs).

Astuce

Le rapport $20 \log_{10}(\mathrm{RMS}_{\mathrm{marche}}/\mathrm{RMS}_{\mathrm{arrêt}})$ est le rapport signal sur bruit de votre installation. En dessous de 10 dB, rapprochez la carte du moteur : aucun modèle ne compensera un capteur mal placé.

Étape 3 : Enregistrer des fichiers WAV sur la microSD

Le format WAV PCM est un en-tête de 44 octets suivi des échantillons bruts en petit-boutiste. On l'écrit soi-même : c'est court et l'on maîtrise exactement le contenu. Le nom du fichier porte la classe, la session et un index, comme exigé pour le dataset du TP4.

```
# audio_wav.py - enregistrement de fenêtres WAV (collecte du dataset)
import audio, time, os, struct
from pyb import LED
from ulab import numpy as np

FE, GAIN_DB = 16000, 24
CLASSE, SESSION = "normal", 1        # à modifier avant chaque enregistrement
DUREE_S, N_FICHIERS = 2, 10          # 10 fichiers de 2 s

RACINE = "/sdcard" if "sdcard" in os.listdir("/") else ""
DOSSIER = RACINE + "/ecotri"
try: os.mkdir(DOSSIER)
except OSError: pass

def entete_wav(n_ech, fe=FE, canaux=1, bits=16):
    octets = n_ech * canaux * bits // 8
    return (b"RIFF" + struct.pack("<I", 36 + octets) + b"WAVE" +
            b"fmt " + struct.pack("<IHHIIHH", 16, 1, canaux, fe,
                                  fe * canaux * bits // 8, canaux * bits // 8, bits)
    +
            b"data" + struct.pack("<I", octets))

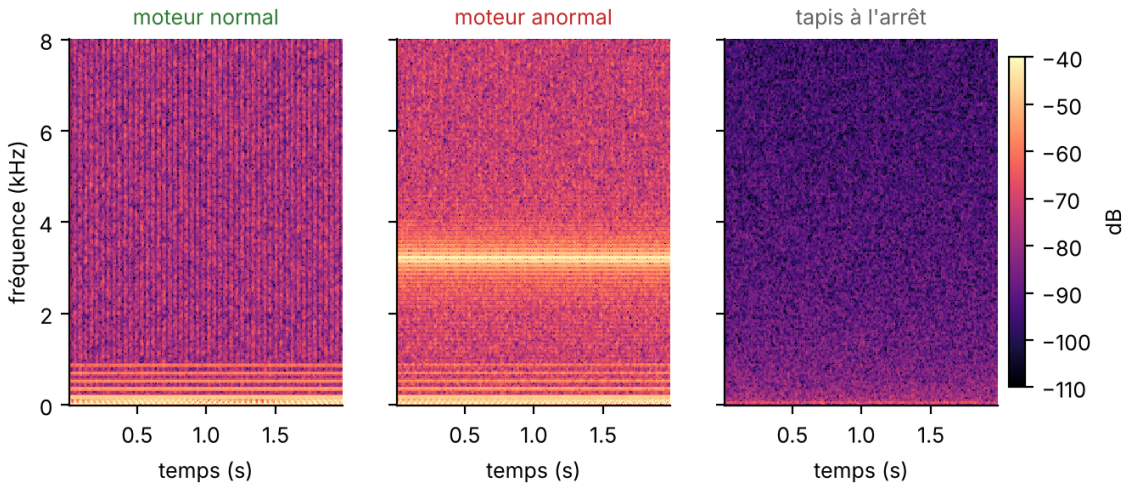
N = FE * DUREE_S
tampon = bytearray(2 * N)            # alloué une seule fois : 64 Ko pour 2 s
pos = 0
plein = False

def rappel(buf):
    global pos, plein
    if plein: return
    n = min(len(buf), len(tampon) - pos)
    tampon[pos:pos + n] = buf[:n]
    pos += n
    if pos >= len(tampon): plein = True

audio.init(channels=1, frequency=FE, gain_db=GAIN_DB, highpass=0.9883)
led = LED(3)
for i in range(N_FICHIERS):
    pos, plein = 0, False
    led.on()
    audio.start_streaming(rappel)
    while not plein: time.sleep_ms(5)
    audio.stop_streaming()
    led.off()
    nom = "%s/%s_s%d_%03d.wav" % (DOSSIER, CLASSE, SESSION, i)
    with open(nom, "wb") as f:
        f.write(entete_wav(N)); f.write(tampon)
    x = np.frombuffer(tampon, dtype=np.int16)
    print(nom, "RMS = %.0f" % np.sqrt(np.mean(x * x * 1.0)))
    time.sleep_ms(300)
os.sync() if hasattr(os, "sync") else None
print("Terminé :", len(os.listdir(DOSSIER)), "fichiers dans", DOSSIER)
```

1. Enregistrez 10 fichiers `normal`, 10 fichiers `arret` (moteur coupé) dans la session 1.
2. Copiez le dossier `ecotri` sur le PC (lecteur de carte, ou disque USB de la carte si la SD est montée en racine).
3. Vérifiez un fichier sur le PC avec le script ci-dessous.

```python
# verif_wav.py - sur le PC
import sys, numpy as np, matplotlib.pyplot as plt
from scipy.io import wavfile
from scipy import signal
fe, x = wavfile.read(sys.argv[1])
print(fe, "Hz,", len(x) / fe, "s, crête", np.abs(x).max())
f, t, S = signal.spectrogram(x.astype(float), fe, nperseg=512, noverlap=352)
plt.pcolormesh(t, f, 10 * np.log10(S + 1e-9), shading="auto", cmap="magma")
plt.xlabel("temps (s)"); plt.ylabel("fréquence (Hz)"); plt.show()
```

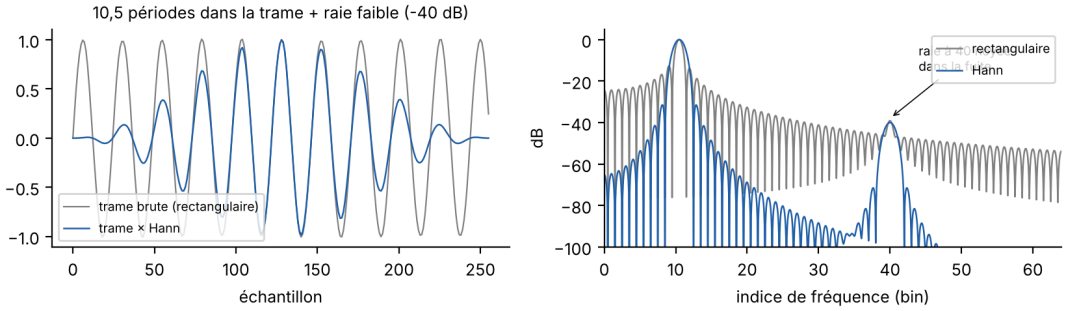


Ce que vous devriez obtenir (ici sur des signaux simulés) : raies stables pour le moteur, bruit diffus à l'arrêt. Comparez avec vos fichiers.

Question

Écoutez un de vos fichiers sur le PC. Entendez-vous autre chose que le moteur (voix, autres machines, chocs des objets) ? Notez-le : ces sources figureront dans la rubrique « Collecte » de la fiche du dataset, et sont des candidats à l'apprentissage par raccourci.

Étape 4 : Calculer un spectre avec ulab



Rappel du cours : sans fenêtre, l'énergie d'une raie fuit sur tout le spectre. On multiplie chaque trame par une fenêtre de Hann avant la FFT.

```
# audio_fft.py - FFT fenêtrée d'une trame de 512 échantillons
import audio, time, math
from ulab import numpy as np

FE, NFFT = 16000, 512
HANN = np.array([math.sin(math.pi * n / NFFT) ** 2 for n in range(NFFT)])
FREQ = np.arange(NFFT // 2 + 1) * FE / NFFT          # fréquence de chaque bin
trame = np.zeros(NFFT); pos = 0; pret = False

def rappel(buf):
    global pos, pret
    if pret: return
    x = np.frombuffer(buf, dtype=np.int16)
    n = min(len(x), NFFT - pos)
    trame[pos:pos + n] = x[:n]
    pos += n
    if pos >= NFFT: pret = True

def spectre_db(x):
    r = np.fft.fft(x * HANN)
    if isinstance(r, tuple): re, im = r               # ulab sans nombres complexes
    else: re, im = r.real, r.imag                     # ulab avec nombres complexes
    re, im = re[:NFFT // 2 + 1], im[:NFFT // 2 + 1]
    return 10 * np.log10(re * re + im * im + 1e-9)

audio.init(channels=1, frequency=FE, gain_db=24, highpass=0.9883)
audio.start_streaming(rappel)
for essai in range(20):
    pos, pret = 0, False
    while not pret: time.sleep_ms(1)
    t0 = time.ticks_us()
    P = spectre_db(trame)
    dt = time.ticks_diff(time.ticks_us(), t0)
    idx = np.argsort(P)[-5:]                          # 5 bins les plus forts
    print("FFT en %d µs ; raies :" % dt, [int(FREQ[i]) for i in idx])
audio.stop_streaming()
```

— Avec un ventilateur ou un moteur, retrouvez-vous des raies régulièrement espacées ? Calculez la vitesse de rotation qui leur correspond.

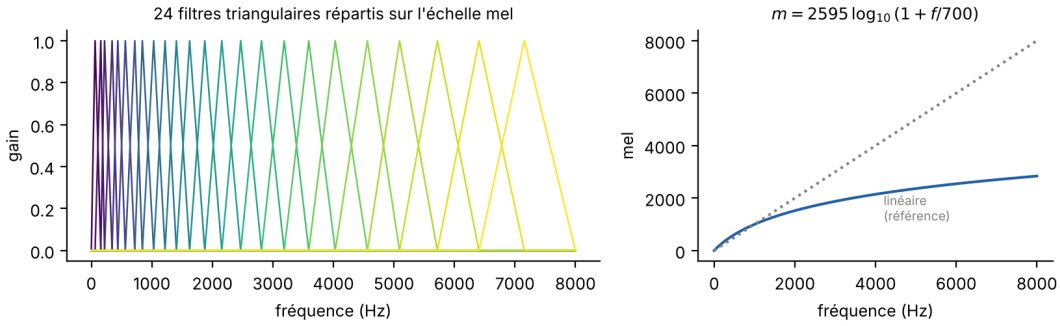
— Jouez un son de 1000 Hz depuis un téléphone (générateur de fréquence) : apparaît-il dans le bon bin ($1000/31{,}25 = 32$) ?

Question

Quelle est la résolution Δf de ce spectre ? Pourquoi ne peut-on pas distinguer 25 Hz de 50 Hz ? Que faut-il changer, et quel en est le coût en mémoire et en temps (mesurez avec `NFFT = 2048`) ?

Étape 5 : Énergies par bande et descripteurs

Calculer 40 bandes mel en MicroPython pur serait lent ; on se contente ici de **8 bandes** à bornes logarithmiques, une version simplifiée du banc mel du cours, et de trois descripteurs temporels. Ces 11 nombres résument une fenêtre de 0,5 s.



Le banc de filtres mel du cours : nos 8 bandes rectangulaires en reprennent l'idée (étroites en basse fréquence, larges en haute fréquence).

```
# audio_bandes.py - descripteurs d'une fenêtre (à placer dans un module bandes.py)
import math
from ulab import numpy as np

FE, NFFT = 16000, 512
HANN = np.array([math.sin(math.pi * n / NFFT) ** 2 for n in range(NFFT)])
BORNES = [0, 125, 250, 500, 1000, 2000, 3000, 5000, 8000]     # Hz, 8 bandes
IDX = [int(f * NFFT / FE) for f in BORNES]
NOMS = ["rms", "crete", "kurt"] + ["b%d" % i for i in range(8)]

def puissance(x):
    r = np.fft.fft(x * HANN)
    re, im = r if isinstance(r, tuple) else (r.real, r.imag)
    return (re * re + im * im)[:NFFT // 2 + 1]

def descripteurs(x):
    """x : ndarray float de longueur multiple de NFFT (ex. 8192 = 0,5 s)."""
    mu = np.mean(x); xc = x - mu
    var = np.mean(xc * xc) + 1e-9
    rms = math.sqrt(var)
    crete = np.max(abs(xc)) / rms
    kurt = np.mean(xc * xc * xc * xc) / (var * var)
    P = np.zeros(NFFT // 2 + 1); n = 0
    for k in range(0, len(x) - NFFT + 1, NFFT):        # moyenne des spectres (Welch)
        P = P + puissance(x[k:k + NFFT]); n += 1
    P = P / n
    bandes = [10 * math.log10(np.sum(P[IDX[i]:IDX[i + 1]]) + 1e-9) for i in range(8)]
    return [20 * math.log10(rms + 1e-9), crete, kurt] + bandes
```

```
# test des descripteurs : un relevé par demi-seconde
import audio, time
from ulab import numpy as np
from bandes import descripteurs, NOMS, FE

N = FE // 2                                            # 8000 éch. = 0,5 s
fen = np.zeros(8192); pos = 0; pret = False
def rappel(buf):
    global pos, pret
    if pret: return
    x = np.frombuffer(buf, dtype=np.int16); n = min(len(x), 8192 - pos)
    fen[pos:pos + n] = x[:n]; pos += n
    if pos >= 8192: pret = True
```

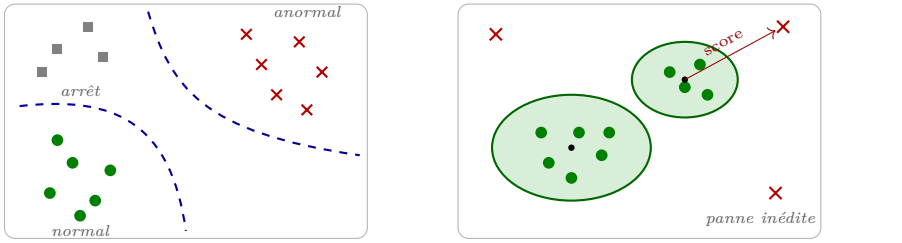
```
audio.init(channels=1, frequency=FE, gain_db=24, highpass=0.9883)
audio.start_streaming(rappel)
print(" ".join("%6s" % s for s in NOMS))
for i in range(30):
    pos, pret = 0, False
    while not pret: time.sleep_ms(2)
    t0 = time.ticks_ms(); d = descripteurs(fen); dt = time.ticks_diff(time.ticks_ms()
    , t0)
    print(" ".join("%6.1f" % v for v in d), " (%d ms)" % dt)
```

Question

Relevez les descripteurs moteur en marche, puis en frottant légèrement une feuille de papier contre le ventilateur (ou en collant un petit morceau de pâte à fixe sur une pale pour créer un balourd). Quels descripteurs changent le plus ? Le kurtosis réagit-il aux chocs comme l'annonce le cours ? Combien de millisecondes prend le calcul : est-ce compatible avec une fenêtre de 0,5 s ?

Étape 6 : Un détecteur d'anomalie appris sur le normal

Classification supervisée (3 classes) **Détection d'anomalies (apprise sur le normal)**



*exige des exemples **étiquetés de chaque panne** ; ne reconnaît que ce qu'il a vu*

*n'exige **que du fonctionnement normal** ; signale « différent », sans dire quoi*

Rappel du cours : la détection d'anomalies n'apprend que le fonctionnement normal et mesure l'écart à ce normal.

Le détecteur le plus simple modélise chaque descripteur normal par une moyenne μ_j et un écart-type σ_j, puis calcule pour une nouvelle fenêtre le score

$$s = \sqrt{\frac{1}{d}\sum_{j=1}^{d}\left(\frac{x_j - \mu_j}{\sigma_j}\right)^2},$$

une distance de Mahalanobis à covariance diagonale. Le seuil est fixé au 99e percentile des scores obtenus sur des fenêtres normales **différentes** de celles de l'apprentissage (validation). L'apprentissage est réalisé **sur la carte**, en trois phases signalées par la LED.

```
# audio_anomalie.py - détecteur d'anomalie embarqué (apprentissage, validation,
    surveillance)
import audio, time, math, json
from pyb import LED
from ulab import numpy as np
from bandes import descripteurs, FE

R, G, B = LED(1), LED(2), LED(3)
```

```
fen = np.zeros(8192); pos = 0; pret = False
def rappel(buf):
    global pos, pret
    if pret: return
    x = np.frombuffer(buf, dtype=np.int16); n = min(len(x), 8192 - pos)
    fen[pos:pos + n] = x[:n]; pos += n
    if pos >= 8192: pret = True

def fenetre():
    global pos, pret
    pos, pret = 0, False
    while not pret: time.sleep_ms(2)
    return np.array(descripteurs(fen))

audio.init(channels=1, frequency=FE, gain_db=24, highpass=0.9883)
audio.start_streaming(rappel)

# Phase 1 : apprentissage du normal (60 fenêtres = 30 s), LED bleue
B.on(); X = [fenetre() for _ in range(60)]; B.off()
X = np.array(X)
mu = np.mean(X, axis=0)
sigma = np.std(X, axis=0) + 1e-3
score = lambda v: math.sqrt(np.mean(((v - mu) / sigma) ** 2))

# Phase 2 : validation sur 40 NOUVELLES fenêtres normales, LED verte
G.on(); sv = sorted(score(fenetre()) for _ in range(40)); G.off()
tau = sv[int(0.99 * (len(sv) - 1))] * 1.1              # P99 + 10 % de marge
print("Seuil tau = %.2f (scores de validation : médiane %.2f, max %.2f)" % (tau, sv[
    len(sv) // 2], sv[-1]))
with open("modele_anomalie.json", "w") as f:            # sauvegarde du « modèle »
    json.dump({"mu": list(mu), "sigma": list(sigma), "tau": tau}, f)

# Phase 3 : surveillance avec lissage « 3 fenêtres sur 5 »
hist = []
while True:
    s = score(fenetre())
    hist = (hist + [s > tau])[-5:]
    alerte = sum(hist) >= 3
    (R.on() if alerte else R.off())
    print("score = %5.2f %s %s" % (s, "*" if s > tau else " ", "ALERTE" if alerte
    else ""))
```

1. Lancez le script moteur en marche et **ne touchez à rien** pendant les phases bleue et verte (50 s).
2. En surveillance, provoquez une anomalie (balourd, frottement) : la LED rouge doit s'allumer après 1 à 2 s.
3. Parlez fort près de la carte, claquez des mains : y a-t-il des fausses alarmes ? Le lissage les filtre-t-il ?
4. Coupez le moteur : que se passe-t-il ? Pourquoi est-ce un problème (indice : « arrêt » n'est pas « panne ») ?

Astuce

Le fichier `modele_anomalie.json` contient tout le modèle : 22 nombres et un seuil. Au prochain démarrage, rechargez-le au lieu de réapprendre. Comparez la taille de ce « modèle » (moins de 1 Ko) à celle d'un réseau de neurones : c'est la référence à battre au TP4.

Étape 7 : Mesurer le détecteur

Mesure	Valeur	Conditions
Temps de calcul des descripteurs (ms)		fenêtre de 0,5 s
Seuil τ obtenu		40 fenêtres de validation
Fausses alarmes en 5 min de fonctionnement normal		avec / sans lissage
Délai de détection d'un balourd (s)		lissage 3/5
Délai de détection d'un frottement (s)		lissage 3/5
Comportement à l'arrêt du moteur		

Tableau à compléter : il constitue la référence embarquée du jalon J4.

Question

Déplacez la carte de 20 cm sans réapprendre : le score augmente-t-il ? Que vous apprend cette expérience sur la notion de **session** vue en cours, et sur la nécessité de découper le dataset par session au TP4 ?

Étape 8 : Relier les mesures au cours

1. Combien d'octets faudrait-il transmettre par jour pour envoyer l'audio brut ? Les 11 descripteurs à chaque fenêtre ? Seulement les alertes ? Retrouvez les ordres de grandeur du tableau du cours.
2. Quelle part du temps CPU d'une fenêtre de 0,5 s le calcul occupe-t-il ? Reste-t-il de la place pour la classification d'images (lecture 5) ?
3. Ce détecteur ignore l'évolution temporelle à l'intérieur d'une fenêtre. Quels défauts pourrait-il manquer ? Quelle représentation du cours les capturerait ?
4. Pour envoyer l'alerte au smartphone, réutilisez la caractéristique DECISION du Tutorial 3 : `decision.write(json.dumps({"src":"moteur","s":s,"alerte":alerte}), send_update=True)`.

Astuce

Pour un programme autonome, enregistrez `bandes.py` sur la carte puis le script principal avec **Tools → Save open script to OpenMV Cam (as main.py)**.

4 Pour aller plus loin

— Remplacez les 8 bandes rectangulaires par 20 filtres triangulaires mel (matrice précalculée) et comparez le temps de calcul et la qualité de détection.

— Utilisez les deux micros (`channels=2`) : la différence entre les canaux donne une information de direction qui aide à rejeter les bruits venant d'ailleurs que du moteur.

— Remplacez l'écart-type par la médiane et l'écart absolu médian (MAD) : le détecteur devient robuste aux valeurs aberrantes de la phase d'apprentissage.

— Testez votre détecteur sur des enregistrements du jeu **MIMII** (ventilateurs, pompes, vannes ; normal et anormal) joués près de la carte.

— Documentation OpenMV des modules `audio` et `ulab` : docs.openmv.io ; exemples *Audio* dans OpenMV IDE.

— R. B. Randall, *Vibration-based Condition Monitoring*, Wiley, 2011, chapitre 5 (diagnostic des roulements, kurtosis).